Classification morphologique de galaxies par réseau de neurones convolutif

Galaxy10 DECaLS : baselines, CNN maison, transfer learning et Grad-CAM

Patrick Yvan Eyike Philémon Vittet Sidi Hamadi Abdelrahman Dahy
Inès Lylia Zerdani

Juillet 2026

Résumé

On classe la morphologie de 17 736 galaxies du relevé DESI Legacy Imaging Surveys (dataset Galaxy10 DECaLS, labels Galaxy Zoo) en 10 classes, à l'aide d'un réseau convolutif. Pour isoler l'apport de la convolution, on compare trois niveaux : baselines sans convolution (régression logistique, MLP), un CNN maison de 1 440 298 paramètres, et un ResNet18 pré-entraîné sur ImageNet. Le meilleur modèle atteint une accuracy de 0,800 et un F1 macro de 0,780 sur le test officiel. On interprète les prédictions par Grad-CAM et on quantifie l'apport de l'augmentation et de la BatchNorm par ablation. Le rapport insiste sur un point de fond : l'équivariance par translation du CNN est une symétrie imposée à l'architecture, dans l'esprit des symétries en physique.

1 Introduction et contexte

La morphologie d'une galaxie (spirale, elliptique, barrée, en fusion) porte une information physique : elle est liée à l'histoire de formation, à l'environnement et à la dynamique du système. À l'échelle des grands relevés, trier des millions de galaxies à la main est impossible. Le projet Galaxy Zoo a mobilisé des centaines de milliers de volontaires pour labelliser des images ; c'est de là que viennent nos étiquettes. La question de ce projet est simple : un réseau convolutif peut-il reproduire ce tri morphologique de façon automatique et fiable ?

On répond en construisant une échelle de trois modèles de complexité croissante, de la baseline aveugle à la structure spatiale jusqu'au réseau pré-entraîné, puis en interprétant ce que le réseau a appris. Historiquement, cette famille de modèles remonte à LeNet (LeCun, 1998), premier CNN entraîné par rétropropagation pour la lecture de chiffres, et à AlexNet (Krizhevsky, 2012), qui a démontré à grande échelle sur ImageNet la supériorité des CNN profonds et relancé tout le domaine.

Travaux liés. La classification morphologique automatique de galaxies par apprentissage profond est aujourd'hui bien établie. Dieleman et al. (2015) ont remporté le concours Galaxy Zoo avec un CNN exploitant explicitement les symétries de rotation des images, confirmant l'intérêt d'encoder les invariances du problème dans l'architecture ou l'entraînement. Le jeu de données Galaxy10, dérivé de Galaxy Zoo et du relevé DESI Legacy Imaging Surveys, sert de banc d'essai standard pour ce type de tâche. Notre contribution n'est pas une nouvelle architecture mais une étude pédagogique contrôlée : isoler l'apport de la convolution par une échelle de modèles, puis interpréter le réseau et mesurer l'effet de chaque ingrédient de régularisation.

2 Données

Galaxy10 DECaLS contient 17 736 images couleur 256×256 (bandes g, r, z du relevé DESI Legacy Imaging Surveys), réparties en 10 classes morphologiques. Le tableau 1 donne les effectifs :

la distribution est fortement déséquilibrée, d'un facteur proche de 8 entre la classe la plus rare (Cigar Shaped, 334 images) et la plus fréquente (Round Smooth, 2 645). Ce déséquilibre est un point central de l'évaluation.

id	classe	n	id	classe	n
0	Disturbed	1 081	5	Barred Spiral	2 043
1	Merging	1 853	6	Unbarred Tight Spiral	1 829
2	Round Smooth	2 645	7	Unbarred Loose Spiral	2 628
3	In-between Round Smooth	2 027	8	Edge-on without Bulge	1 423
4	Cigar Shaped Smooth	334	9	Edge-on with Bulge	1 873

Table 1 – Les 10 classes de Galaxy10 DECaLS et leurs effectifs.

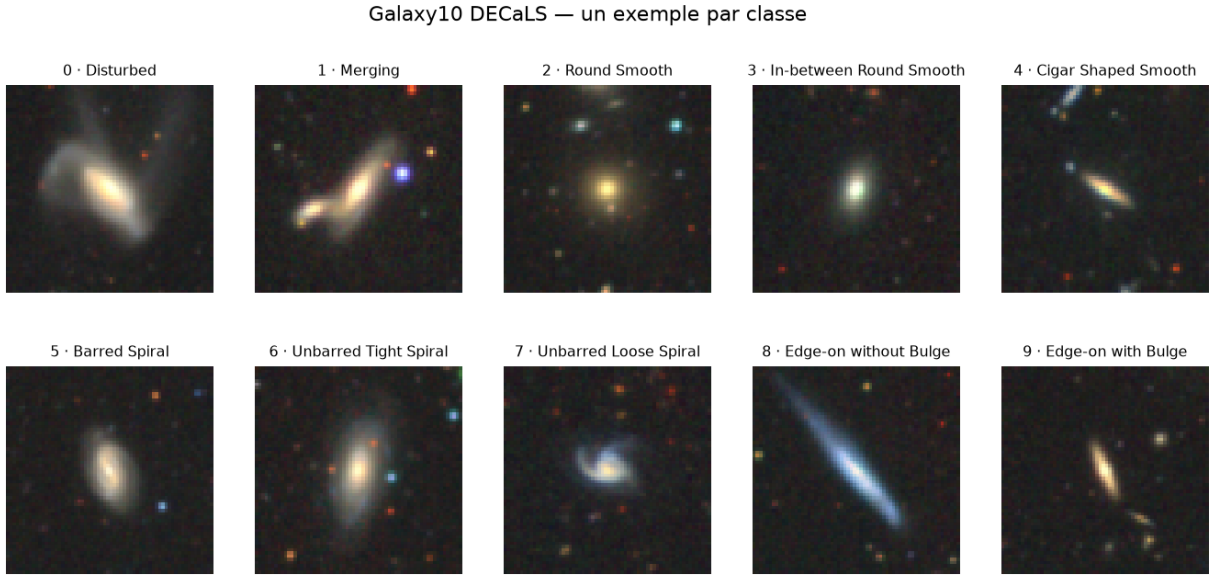


Figure 1 – Un exemple par classe. Les classes voisines sont visuellement proches : spirales serrées contre lâches, rondes contre cigare, edge-on avec ou sans bulbe.

Découpage. Le dataset fournit déjà un `train` (16 000) et un `test` (1 800). On garde le test officiel comme test final, jamais utilisé pendant le développement, et on tire une validation stratifiée de 15 % dans le train (seed figé). Toutes les décisions (choix de modèle, hyperparamètres) se prennent sur la validation. Ce protocole évite deux fuites classiques : décider sur le test, et augmenter les données avant le découpage.

3 Éléments de théorie

3.1 Du perceptron au réseau profond

Un réseau de neurones enchaîne des couches affines $z = Wx + b$ suivies d'une non-linéarité, typiquement la ReLU $\sigma(z) = \max(0, z)$. La dernière couche produit un vecteur de scores transformé en probabilités par la softmax $p_k = e^{z_k} / \sum_j e^{z_j}$. On entraîne en minimisant l'entropie croisée

$$\mathcal{L} = -\sum_k y_k \log p_k = -\log p_y, \tag{1}$$

qui n'est autre que la log-vraisemblance négative du bon label sous le modèle : minimiser l'entropie croisée revient à maximiser la vraisemblance des données. Les gradients se calculent par

rétropropagation (application organisée de la règle de la chaîne, de la loss vers les poids), et on met à jour les paramètres par descente de gradient stochastique ou, ici, par Adam. Le learning rate est l'hyperparamètre le plus sensible : trop grand, la loss diverge ; trop petit, rien n'apprend.

Concrètement, la rétropropagation calcule $\partial \mathcal{L}/\partial W$ pour chaque couche en propageant le gradient de la sortie vers l'entrée : chaque couche reçoit le gradient amont, le combine avec la dérivée locale de son opération, et transmet le résultat à la couche précédente. Adam raffine la descente de gradient en gardant deux moyennes mobiles par paramètre, celle du gradient (moment) et celle de son carré, ce qui adapte le pas à chaque coordonnée et rend l'entraînement robuste au choix initial du learning rate. On y ajoute un scheduler cosine qui décroît progressivement le pas, et un early stopping qui arrête l'entraînement quand le F1 macro de validation cesse de progresser, pour éviter le sur-apprentissage.

3.2 La convolution 2D

Une couche convolutive fait glisser un même petit filtre sur toute l'image et calcule, en chaque position, une somme pondérée locale :

$$(I * K)_{i,j} = \sum_m \sum_n I_{i+m,\, j+n}\, K_{m,n}. \tag{2}$$

C'est exactement la convolution discrète familière en traitement du signal, le filtre jouant le rôle de réponse impulsionnelle : chaque canal de sortie est un détecteur d'un motif local (bord, tache, orientation). Deux propriétés en découlent.

Partage de poids. Le même filtre sert partout dans l'image. Le nombre de paramètres ne dépend donc pas de la taille de l'image mais seulement de la taille du filtre et du nombre de canaux, ce qui rend l'apprentissage statistiquement efficace et régularise le modèle.

Équivariance par translation et symétries. Comme le filtre est appliqué de façon identique partout, translater un motif dans l'image translate d'autant sa réponse : $f(T_v\, x) = T_v\, f(x)$. Le CNN encode donc *a priori* une symétrie du problème dans son architecture, plutôt que de devoir l'apprendre. C'est le même principe qu'en physique, où l'on impose les symétries d'un système (invariances par translation, rotation) avant de résoudre les équations. Une galaxie n'ayant pas de position ni d'orientation privilégiée dans le ciel, ces invariances sont physiquement justifiées, et l'on complète l'équivariance par translation du CNN avec une augmentation par rotations et symétries (section 4).

3.3 Architecture et champ réceptif

On empile des blocs `Conv` $\rightarrow$ `BatchNorm` $\rightarrow$ `ReLU` $\rightarrow$ `MaxPool`. Le pooling réduit la résolution spatiale et augmente le champ réceptif : les couches profondes voient une portion de plus en plus grande de l'image d'origine, ce qui construit une hiérarchie de features, des bords locaux aux textures puis aux formes globales (bras spiraux, barre, compagnon de fusion). Quantitativement, chaque convolution 3×3 élargit le champ réceptif de 2 pixels et chaque MaxPool le double : après nos quatre blocs, un neurone de la dernière carte voit une portion de l'image d'entrée bien plus large que 3×3, ce qui lui permet de répondre à une structure morphologique entière plutôt qu'à un simple bord.

3.4 Dimensionnement et nombre de paramètres

Notre CNN prend une entrée $3 \times 64 \times 64$. Une convolution 3×3 avec padding 1 conserve la taille spatiale ; chaque MaxPool la divise par deux. Le tableau 2 suit les dimensions bloc par bloc.

couche	sortie (C, H, W)	poids conv (sans biais)
entrée	$3 \times 64 \times 64$	–
bloc 1 : Conv($3 \to 32$)+pool	$32 \times 32 \times 32$	$3 \cdot 32 \cdot 9 = 864$
bloc 2 : Conv($32 \to 64$)+pool	$64 \times 16 \times 16$	$32 \cdot 64 \cdot 9 = 18\,432$
bloc 3 : Conv($64 \to 128$)+pool	$128 \times 8 \times 8$	$64 \cdot 128 \cdot 9 = 73\,728$
bloc 4 : Conv($128 \to 256$)+pool	$256 \times 4 \times 4$	$128 \cdot 256 \cdot 9 = 294\,912$
AdaptiveAvgPool(4×4)	$256 \times 4 \times 4$	–
FC $4096 \to 256$	256	$4096 \cdot 256 + 256 = 1\,048\,832$
FC $256 \to 10$	10	$256 \cdot 10 + 10 = 2\,570$

Table 2 – Dimensions et paramètres du CNN maison. Les convolutions sont sans biais (la BatchNorm fournit le décalage). En ajoutant les 2 paramètres par canal de BatchNorm ($64 + 128 + 256 + 512 = 960$), le total est de 1 440 298 paramètres, dominé à 73 % par la première couche dense. L'`AdaptiveAvgPool` fixe la taille de la tête, ce qui garde ce compte identique en 64 comme en 128 px.

3.5 Régularisation

Un réseau à plus d'un million de paramètres sur quelques milliers d'images sur-apprend facilement. On combine trois régularisations. Le *dropout* éteint aléatoirement des unités pendant l'entraînement. La *BatchNorm* normalise les activations par mini-lot, ce qui stabilise et accélère l'apprentissage. La *data augmentation* enrichit artificiellement le train avec des transformations qui préservent le label : ici rotations 0–360°, symétries horizontales et verticales, léger jitter de luminosité. Comme souligné plus haut, ces transformations sont des invariances physiques du problème et non un simple bruit.

3.6 Transfer learning

Plutôt que d'apprendre de zéro, on réutilise un ResNet18 pré-entraîné sur ImageNet. Le ResNet introduit les connexions résiduelles $y = \mathcal{F}(x) + x$, qui laissent le gradient circuler et permettent d'entraîner des réseaux profonds. On remplace la dernière couche par 10 sorties et on procède en deux temps : *feature extraction* (backbone gelé, on n'entraîne que la tête) puis *fine-tuning* (tout dégelé, learning rate plus faible).

4 Pipeline expérimental

Les images sont redimensionnées en 64 px (baselines et CNN) et 128 px (ResNet), puis normalisées canal par canal avec des statistiques calculées *sur le train uniquement*. L'augmentation n'est appliquée qu'au train, jamais à la validation ni au test. Le déséquilibre est traité en pondérant l'entropie croisée par l'inverse des fréquences de classe, et l'évaluation privilégie le F1 macro (qui traite toutes les classes à égalité) plutôt que la seule accuracy. L'entraînement utilise Adam, un scheduler cosine et un early stopping sur le F1 macro de validation. Le code est organisé en un package Python réutilisable et un notebook auto-portant ; les runs tournent indifféremment en local (Mac, backend MPS) et sur Colab (GPU T4), avec détection automatique du device.

Protocole et reproductibilité. Le tirage de validation, l'initialisation des poids et le mélange des lots sont contrôlés par une graine fixe (42). La validation représente 15 % du train (soit environ 2 400 images), le test officiel 1 800 images. Les lots font 128 images, l'optimiseur est Adam avec un scheduler cosine, et l'early stopping a une patience de 8 epochs. Les statistiques de normalisation, la meilleure architecture et les hyperparamètres sont figés à partir de la validation, puis appliqués une seule fois au test. L'ensemble (chargement, découpage, entraînement, métriques, figures, Grad-CAM) est déterministe à graine donnée, ce qui rend les résultats reproductibles.

5 Détails d'implémentation

Le cœur du code tient en peu de lignes. Un bloc convolutif enchaîne une convolution sans biais, une BatchNorm, une ReLU et un pooling ; le CNN empile quatre de ces blocs puis une tête dense.

```
def _conv_block(in_c, out_c, use_bn=True):
    layers = [nn.Conv2d(in_c, out_c, 3, padding=1, bias=not use_bn)]
    if use_bn:
        layers.append(nn.BatchNorm2d(out_c))
    layers += [nn.ReLU(inplace=True), nn.MaxPool2d(2)]
    return nn.Sequential(*layers)
```

Grad-CAM se réduit à deux hooks (pour capturer les activations et les gradients de la dernière convolution) et à une combinaison pondérée suivie d'une ReLU.

```
out = model(x) # x : [1, 3, H, W]
model.zero_grad()
out[0, class_idx].backward()
weights = gradients.mean(dim=(2, 3), keepdim=True) # importance par canal
cam = F.relu((weights * activations).sum(1, keepdim=True))
cam = F.interpolate(cam, size=x.shape[-2:], mode="bilinear")
```

6 Expériences et résultats

6.1 L'échelle baseline → CNN → ResNet

Le tableau 3 et la figure 2 résument l'apport de la convolution. Les baselines, aveugles à la structure spatiale, restent médiocres. Le CNN maison progresse nettement, et le transfer learning ajoute encore un cran.

modèle	accuracy	F1 macro
Régression logistique (pixels)	0,280	0,259
MLP 2 couches (pixels)	0,439	0,404
CNN maison	0,705	0,683
ResNet18 (fine-tuné)	0,800	0,780

Table 3 – Performances sur le test officiel (jamais utilisé pendant le développement).

6.2 Tuning et courbes d'apprentissage

Le learning rate est réglé sur la validation ; la meilleure valeur est $\text{lr} = 3 \times 10^{-4}$. Les courbes de la figure 3 montrent l'évolution de la loss et du F1 macro de validation, avec l'epoch retenue par early stopping.

6.3 Analyse par classe et confusions

La matrice de confusion (figure 4) et le F1 par classe (figure 5) montrent où le modèle se trompe. Les classes lisses et les galaxies edge-on sont les mieux reconnues : Round Smooth obtient le meilleur F1, car sa morphologie est nette et peu ambiguë. À l'inverse, la classe Disturbed est la plus difficile ; les galaxies perturbées recouvrent visuellement les fusions et forment une catégorie hétérogène, ce qui relève d'une ambiguïté de définition plus que d'un simple manque d'exemples. Les spirales serrées et lâches se confondent aussi entre elles, et la classe rare Cigar Shaped reste fragile. Les confusions se concentrent donc, comme attendu, entre classes morphologiquement proches.

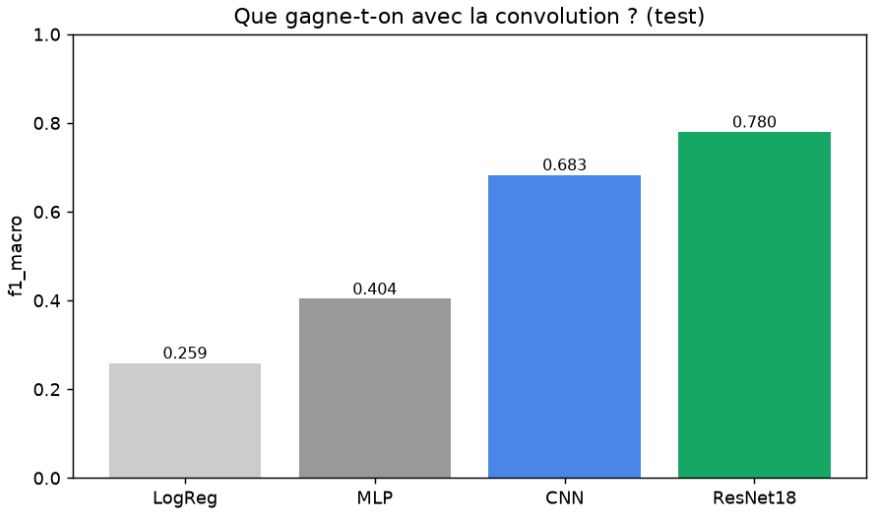


Figure 2 – F1 macro sur le test, des baselines au ResNet. La convolution, puis le pré-entraînement, expliquent l'essentiel du gain.

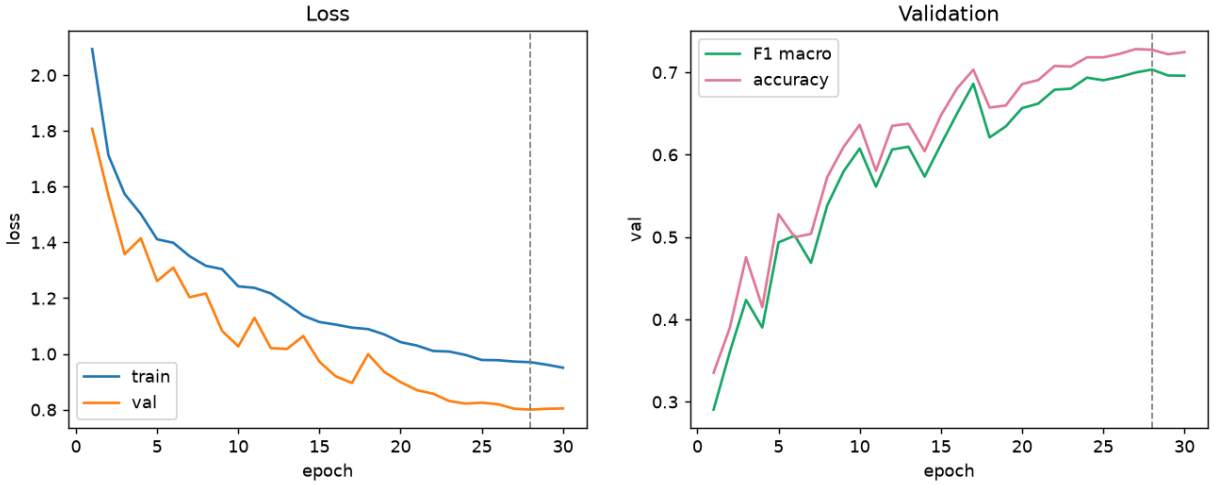


Figure 3 – Courbes d'apprentissage du CNN maison (loss train/val et métriques de validation). La ligne verticale marque l'epoch du meilleur F1 macro.

6.4 Interprétation par Grad-CAM

Grad-CAM pondère les feature maps de la dernière convolution par le gradient moyen du score de la classe, applique une ReLU, puis rééchelonne à la taille de l'image ; on obtient une carte de chaleur des régions qui pilotent la prédiction. Formellement, pour la classe c et les feature maps A^k de la dernière convolution, on calcule les poids $\alpha_k^c = \frac{1}{Z}\sum_{i,j} \partial y^c / \partial A_{i,j}^k$ (moyenne spatiale du gradient), puis la carte $L^c = \mathrm{ReLU}\big(\sum_k \alpha_k^c A^k\big)$, rééchelonnée à la taille de l'image. La figure 6 montre des cas corrects et des erreurs. Sur les bonnes prédictions, le réseau se concentre sur les structures pertinentes (bras spiraux, barre, compagnon de fusion) ; sur les erreurs, la carte révèle parfois un appui sur un objet voisin ou un artefact.

6.5 Ablations

On retire un ingrédient à la fois pour mesurer son apport (F1 macro sur le test). Le CNN complet atteint 0,683, la variante sans augmentation 0,687 et la variante sans BatchNorm 0,719. Contrairement à l'intuition, l'augmentation n'apporte quasiment rien dans notre régime, et retirer la BatchNorm améliore même légèrement le F1 de test. Deux lectures cohérentes se dégagent. À 64

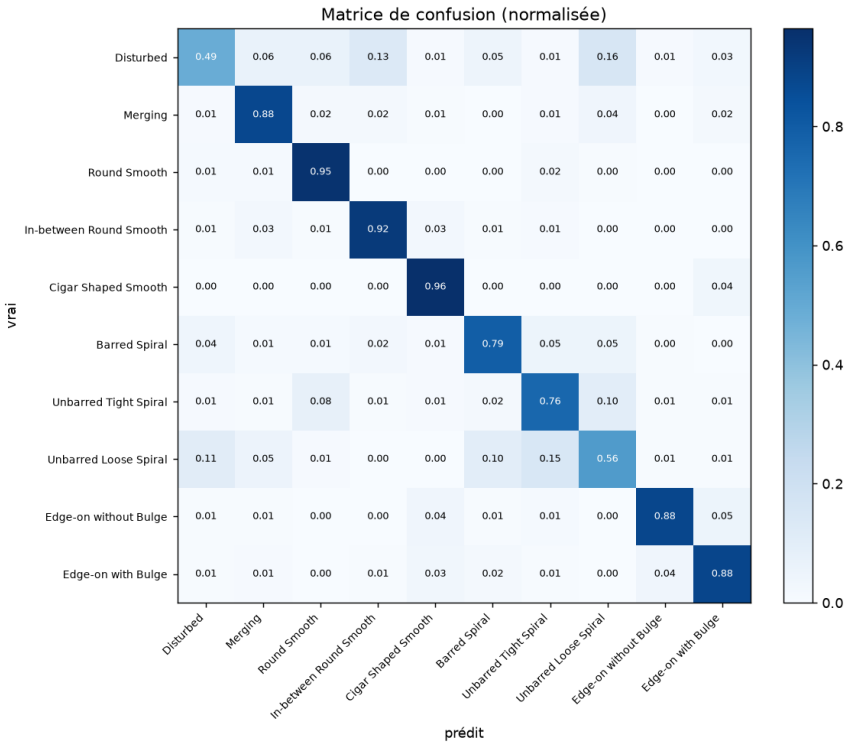


Figure 4 – Matrice de confusion normalisée par ligne (ResNet18, test).

px et 30 epochs, le CNN ne sur-apprend pas fortement : la régularisation par augmentation a donc peu d'effet, elle est surtout utile quand l'overfitting est marqué. Et un seul tirage suffit à noyer un écart modeste dans le bruit ; une ablation rigoureuse devrait moyenner sur plusieurs graines. La BatchNorm garde par ailleurs son intérêt principal, accélérer et stabiliser la convergence, ce que confirment les courbes d'apprentissage. Ce résultat honnête rappelle que l'apport d'un ingrédient de régularisation dépend du régime d'entraînement et n'est pas acquis d'avance.

7 Limites

Trois limites principales. Les labels de Galaxy Zoo sont bruités : ils agrègent des votes de volontaires et les frontières entre classes voisines sont floues, ce qui plafonne mécaniquement la performance atteignable. Le déséquilibre pénalise les classes rares malgré la pondération de la loss. Enfin, le coût d'entraînement limite l'exploration : on a travaillé en 64 et 128 px plutôt qu'en 256, et l'espace des hyperparamètres n'a été balayé que partiellement.

Plusieurs pistes prolongeraient naturellement ce travail. On pourrait entraîner en 256 px pour exploiter la résolution native des images, remplacer la pondération de la loss par un sur-échantillonnage des classes rares ou une focal loss, et estimer l'incertitude des prédictions (par exemple par dropout au test) pour signaler les cas ambigus plutôt que de forcer une décision. Comparer plusieurs backbones pré-entraînés et exploiter explicitement l'invariance par rotation dans l'architecture (couches équivariantes) sont deux autres directions cohérentes avec l'esprit du projet.

8 Conclusion

L'échelle baseline → CNN → ResNet répond à la question de départ : la convolution, puis le transfer learning, apportent l'essentiel de la performance, et le meilleur modèle reproduit correctement le tri morphologique de Galaxy Zoo. Grad-CAM confirme que le réseau s'appuie sur les

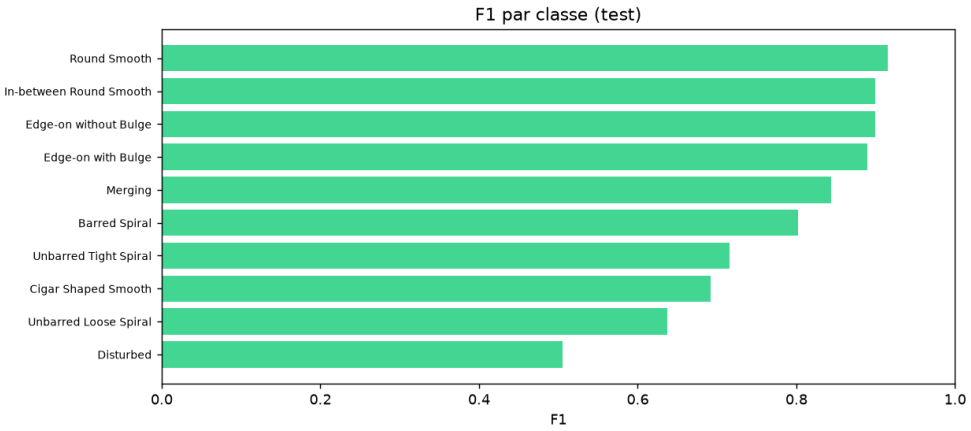


Figure 5 – F1 par classe sur le test. Les classes rares et ambiguës sont en bas.

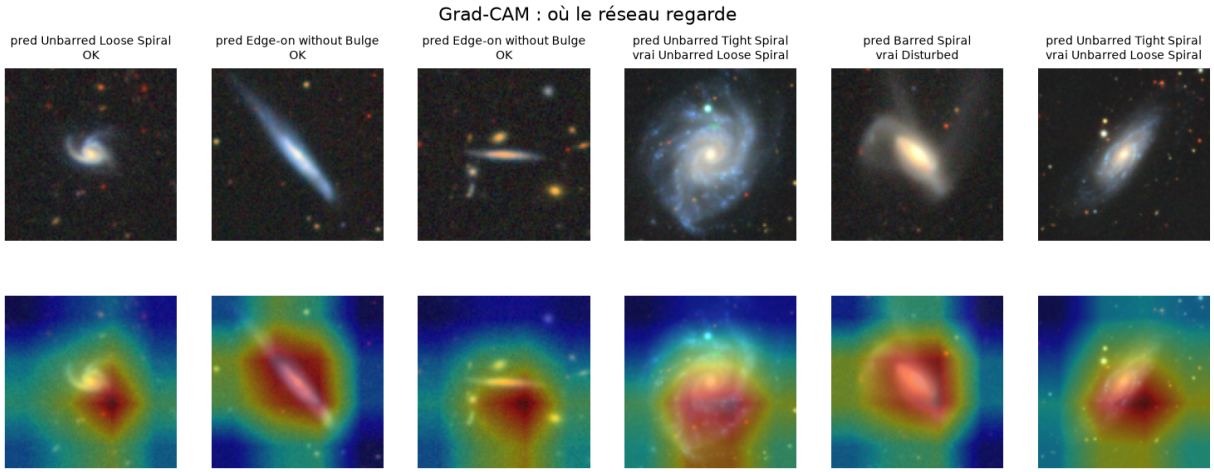


Figure 6 – Grad-CAM sur le ResNet18 (ligne haute : image ; ligne basse : carte superposée). Prédictions correctes puis erreurs.

structures morphologiques attendues, et les ablations quantifient l'apport de chaque ingrédient. Le point conceptuel à retenir est que le succès du CNN sur des images tient largement à une symétrie imposée par construction, l'équivariance par translation, prolongée par une augmentation qui encode les invariances physiques du problème.

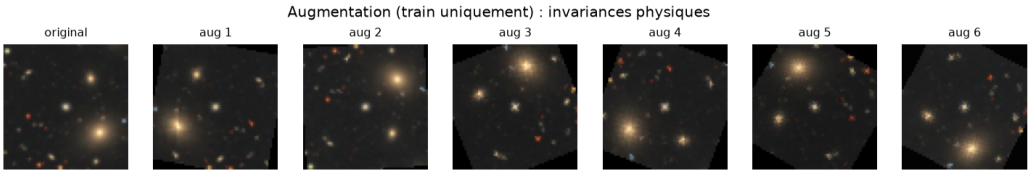


Figure 7 – Effet de l'augmentation sur une même galaxie (rotations, symétries, jitter). Le label est préservé, l'apparence varie.